Software Coding & Software Testing

Readable code · the five types of testing · black-box and white-box test design · cyclomatic complexity · test-first development, stubs, mocks and defects



Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the Next Generation of IT Pros.

LO3 (Learning Outcome 3): characterise unit, integration, system, user acceptance and performance testing

© 2026 Cyber.SoHo — Educational Hub · all teams, products and data are fictional · governed by the Cyber.SoHo Legal Notice

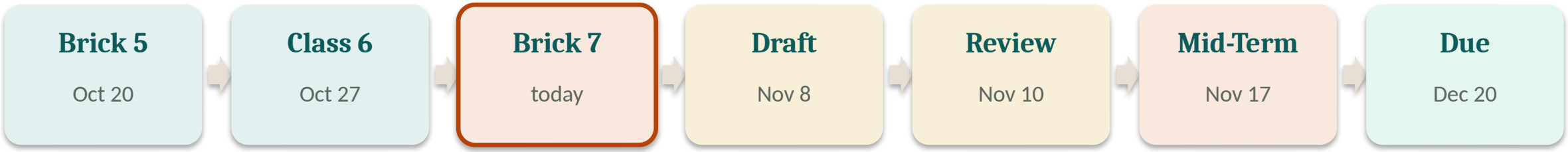
The hour, minute by minute

Part A	`0:00–0:11`	Lesson 1 — Code other people can read
	`0:11–0:15`	✋ Active Break — Try-it: rename this code
Part B	`0:15–0:26`	Lesson 2 — Testing fundamentals and the five types of testing
	`0:26–0:30`	👁️ Active Break — Visualize-it: draw the V-model and place the five types
Part C	`0:30–0:36`	Lesson 3 — Black-box test design
	`0:36–0:41`	Lesson 4 — White-box test design: coverage, flow graphs and **V(G)** (cyclomatic complexity)
	`0:41–0:45`	🎯 Active Break — Fun-quiz: three questions, vote with your hands
Part D	`0:45–0:56`	Lesson 5 — Test-Driven Development, test doubles, debugging and defects
	`0:56–1:00`	🧘 Active Break — Zen-break, then a flash-card sprint

Then: break (1:00–1:10) · Studio with Lab 07 (1:10–2:50) · wrap-up and exit ticket (2:50–3:00)

Where we are — Brick 7 today

- **LO3** (Learning Outcome 3): characterise unit, integration, system, user acceptance and performance testing
- Brick 5 said what the product must do; Brick 7 proves that the code does it
- Studio today: Lab 07 — first code, ≥ 6 unit tests, coverage, black-box tables, **V(G)** (cyclomatic complexity), Test Plan v1
- Due Sunday, December 20, 23:59 · first version `block7-draft` by Sunday, November 8 for feedback
- Mid-Term, Tuesday, November 17: Part B computes **V(G)** (cyclomatic complexity), coverage and defect density; Part C designs test cases



Why readable code is an engineering requirement

- Code is written once and read many times — by the reviewer, the teammate, the future you
- Understanding existing code took about 58 % of developers' time (Xia et al., 2018)
- Unreadable code is slow to change, and misunderstood code gets new defects
- **Coding standard**: a team's written rules for names, layout, comments and language features
- **Style guide**: a published standard for a language — **PEP 8** (Python Enhancement Proposal 8, the official style guide for Python code) for Python, the Google style guides

1

Readable code

0:00–0:02

Part A of the hour

Names that say what they mean

- **Identifier**: the name of a variable, function, class or constant — it should say what the thing is or does
- Variables: nouns with units (`days_late`) · functions: verbs (`compute_late_fee`) · Booleans: questions (`is_overdue`) · classes: nouns (`Loan`)
- **PEP 8** (Python Enhancement Proposal 8, the official style guide for Python code): `snake_case` functions and variables · `CapWords` classes · `UPPER_CASE` constants · 4 spaces · ≤ 79 characters per line
- **Magic number**: a literal whose meaning is not written (`0.5`, `10`) → replace it with a **named constant** (`DAILY_LATE_FEE = 0.50`)
- Avoid: single letters, private abbreviations (`clc_fee`), false names (`loan_list` for a dictionary), numbered names (`data2`)

1

Readable code

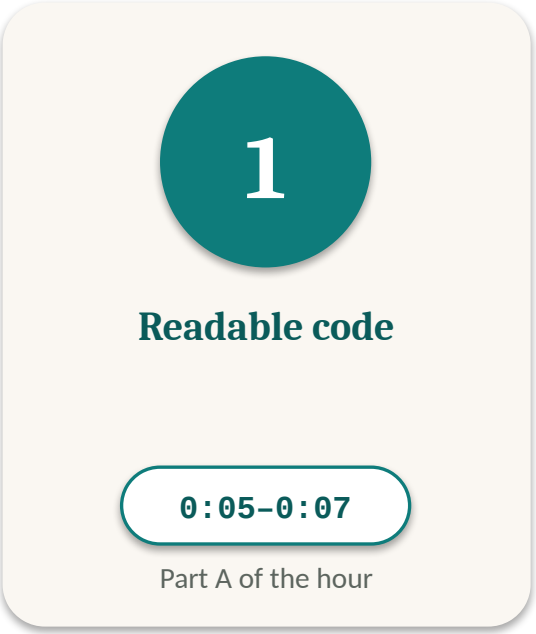
0:02–0:05

Part A of the hour



Comments that explain why, docstrings that explain what

- **Comment**: ignored by the interpreter; explains *why* — a reason, a constraint, the requirement (`# FR-06: fee capped at $10.00 per loan`)
- Useless: repeats *what* the code says (`x = x + 1  # add one to x`) · Harmful: describes old behaviour
- **Docstring**: the first string in a module, function or class; shown by `help()` and editors
- **PEP 257** (Python Enhancement Proposal 257, the docstring conventions): one summary line ending with a period, then parameters, result, exceptions, requirement
- The comment before every line in our demos is a teaching format — production code comments the reasons



Small functions that do one thing

- One function, one task, one level of detail — if its name needs “and”, split it
- **Guard clause**: return early for a special case instead of nesting `if` inside `if`
- Short enough to read without scrolling · no copied blocks: the **DRY** (Don't Repeat Yourself) principle
- **Linter**: checks code without running it — Ruff or Flake8 (Python), ESLint (JavaScript), Checkstyle (Java)
- **Formatter**: rewrites the layout automatically — Black (Python) · run both before every commit

1

Readable code

0:07–0:08

Part A of the hour



Code review, pair programming and clean commits

- **Code review**: a teammate reads a change before the merge — correct? readable? tested? follows design and standard?
- **Pull request**: a request to merge one branch into another; change, comments and approval stay together
- Reviews find unclear names, missing cases, misread requirements — and spread knowledge; keep changes small
- **Pair programming**: **driver** types, **navigator** checks each line and thinks ahead; swap every 10 minutes
- Utah experiment (Cockburn & Williams, 2000): about 15 % more time, about 15 % fewer defects
- **Commit hygiene**: one logical change, tests passing, summary ≈ 50 characters in the imperative, blank line, why

1

Readable code

0:08–0:11

Part A of the hour



✋ Active Break — Try-it: Rename this code

ACTIVE BREAK · TRY-IT · 0:11–0:15

In pairs, three minutes: rewrite the seven lines below so that a stranger can read them — rename every identifier and replace every magic number with a named constant. One pair reads its version aloud; then the professor shows the reviewed version from the real-life example.

```
def f(d, r, x):
    # calculate
    if (r - d).days > 0:
        y = (r - d).days * 0.5
        if y > 10: y = 10
        return y
    else: return 0
```

Real-life example — ClubLend (Team Owl)

THE TASK

Rewrite a working but unreadable late-fee function so that it follows the team's coding standard, check it with a five-item review checklist, prove that its behaviour has not changed, and record the change in one clean commit.

HOW IT RELATES TO THE LESSON

Applies Lesson 1 directly: identifiers, named constants instead of magic numbers, a docstring with the requirement, one task per function, a code review in a pull request, and commit hygiene.

WHERE ITS DELIVERABLES GO — LABS ALREADY SCHEDULED

- **Lab 07, Step 2 — the rules first, in one cohesive module; four Cs: Clear —** The readable rule module, with named constants and docstrings
- **Lab 07, Step 2 (pairs review each other) and the four Cs: Clear —** The review checklist, posted as a pull-request comment
- **Lab 07, Step 1 ("a commit message that says so") and Step 9 (commit, tag, push) —** Commit messages that say what changed and why
- **Lab 07 file, section 0 — tools and techniques —** The linter or formatter chosen by the team, with one reason

Run it and watch it — Demo 1 and the videos

THE PROGRAM — RUN IT STEP BY STEP

- **Demo 1 — a small code reviewer for ClubLend's late fee** — Reviews two versions of ClubLend's late-fee function — Maya's first version and the reviewed one — against five rules of the team's coding standard, prints every finding, and then runs both versions on three loans to prove that the behaviour did not change.
- Open the module, Lesson 1: ▶️ Start, ❚❚ Pause, step through, read the trace; download `d1_review.py`

THE BEST SHORT VIDEOS FOR THIS LESSON



Naming Things in Code

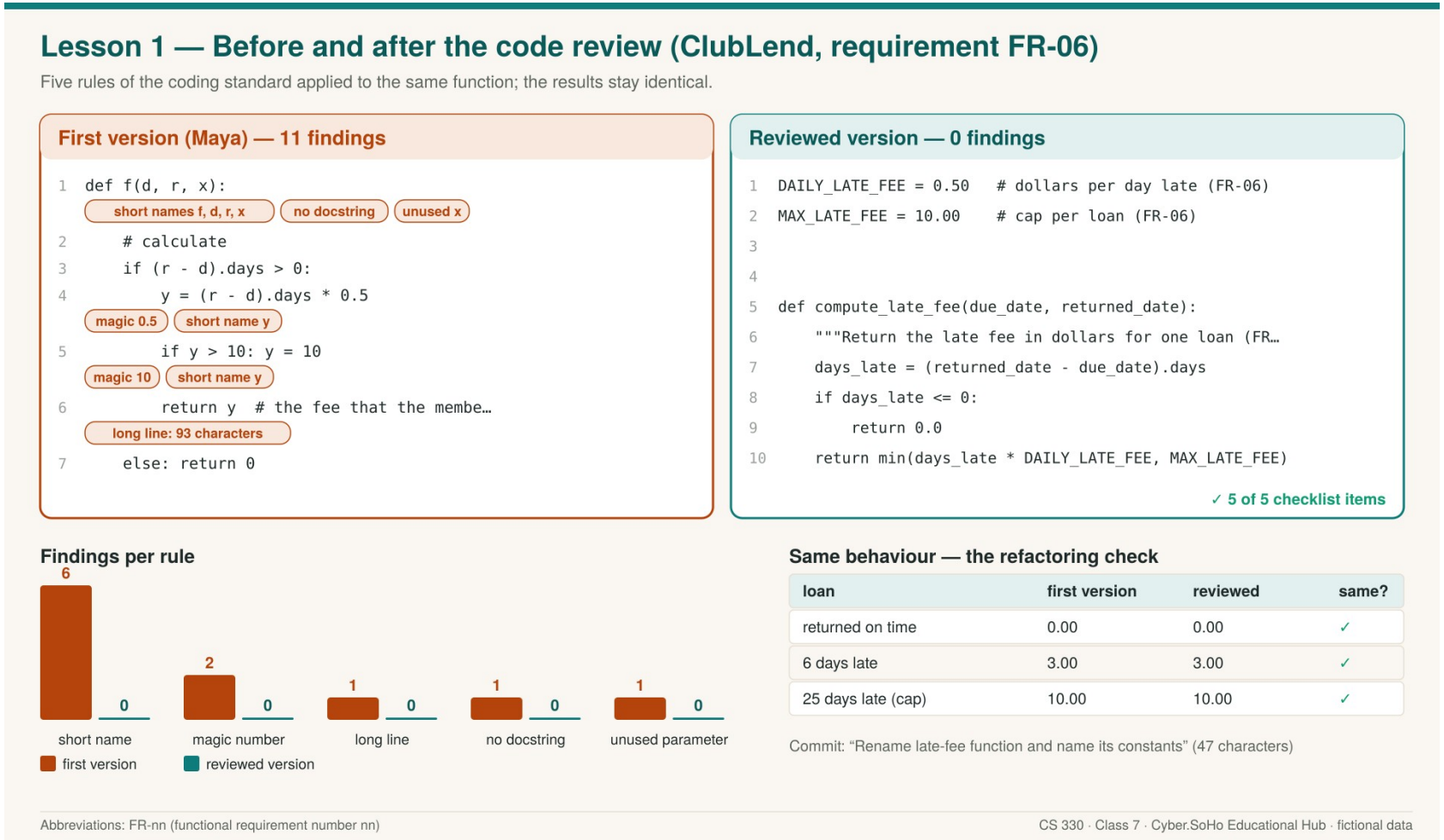
CodeAesthetic · 7:25



How to create a pull request in 4 min | GitHub for Beginners

GitHub · 3:44

Figure 1 — Before and after the code review



ClubLend's late-fee function before and after the review: eleven findings of five rules become zero, and the same three loans give the same fees — readability changed, behaviour did not.

What testing can and cannot do

- **Software testing**: running the program with selected inputs and comparing actual with required behaviour
- **Error** (human mistake) → **defect** (fault, bug: the wrong code) → **failure** (observable wrong behaviour)
- Example: “at least 10” read as “more than 10” → `>` instead of `>=` → cancellation refused at exactly 10 minutes
- Tests can show defects, never prove their absence (Dijkstra, 1970) — so choose the sample well
- The later a defect is found, the more it costs

2

The five types of testing

0:15–0:17

Part B of the hour



Verification and validation

- **Verification**: are we building the product right? — against the specification (reviews, unit, integration, system tests)
- **Validation**: are we building the right product? — against the users' needs (user acceptance tests, prototypes, trial use)
- Boehm, 1979: the two questions
- All requirements met ≠ users satisfied — a product can pass verification and fail validation

2

The five types of testing

0:17–0:19

Part B of the hour

The five types of testing (1/2)

- **Unit**: one function or class alone · developers · every change · milliseconds
- **Integration**: interfaces between parts · big-bang or incremental (top-down with **stubs**, bottom-up with **drivers**)
- Mars Climate Orbiter (**NASA** (National Aeronautics and Space Administration), 1999): pound-force seconds sent, newton-seconds expected — an interface defect

2

The five types of testing

0:19–0:23

Part B of the hour

The five types of testing (2/2)

- **System**: the whole product against the **SRS** (Software Requirements Specification) · staging environment · independent testers
- **User acceptance** (**UAT** (user acceptance testing)): real users confirm it supports their tasks — validation · alpha and beta
- **Performance**: response time and throughput under load — **load test** (expected peak), **stress test** (beyond it)
- The first four are levels; performance is a non-functional type · **regression testing** re-runs tests after every change

2

The five types of testing

0:19–0:23

Part B of the hour

The V-model: each level checks one specification

- Detailed design ↔ unit tests · architecture ↔ integration tests · **SRS** (Software Requirements Specification) ↔ system tests · users' needs ↔ acceptance tests
- Tests are designed on the left, when each document is written, and run on the right
- Brick 5's verification lines and alternative flows are already system and acceptance test cases

2

The five types of testing

0:23–0:24

Part B of the hour

Test strategy, test plan, entry and exit criteria

- **Test strategy**: the general approach — types of testing, order, techniques, automation, measures
- **Test plan**: for one version — **scope** (in and out), the five types with who and when, tools, environments
- **Entry criteria**: conditions to start a test activity (the requirement has a verification line, the code is committed)
- **Exit criteria**: measurable conditions to finish and release (all unit tests pass, branch coverage ≥ 80 %, no open severity-1 defect)
- Full content of test documentation: **ISO/IEC/IEEE 29119** (the joint software-testing standard of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers) part 3 · Brick 7: six parts on one page

2

The five types of testing

0:24–0:26

Part B of the hour



👁 Active Break — Visualize-it: Draw the V-model and place the five types

ACTIVE BREAK · VISUALIZE-IT · 0:26–0:30

On paper, from memory: draw the V-model — the users' needs, the requirements (the **SRS** (Software Requirements Specification)), the architecture and the detailed design down the left side, the code at the bottom — and write the five types of testing on the right side, each opposite the document it checks. Where does performance testing go? Compare with your neighbour, then look at Figure 2.

Real-life example — HabitHub (Team Wren)

THE TASK

Write the one-page Test Plan v1 for HabitHub's streak story — scope, the five types of testing with who and when, tools, entry and exit criteria, defect process — then use the exit criteria, with the team's real numbers, to decide whether the version may be tagged.

HOW IT RELATES TO THE LESSON

Applies Lesson 2: the five types of testing of **LO3** (Learning Outcome 3), verification and validation, the V-model, and a test plan with measurable entry and exit criteria.

WHERE ITS DELIVERABLES GO — LABS ALREADY SCHEDULED

- **Lab 07, Step 5 — template section 1 —** Test Plan v1 — the six-part table
- **Lab 07, Step 9 (commit, tag, push) and section 8 (the numbers) —** Exit criteria checked with numbers before tagging
- **Lab 08 (summary only — created separately): those four types are run there at a small scale —** Who and when for integration, system, acceptance and performance testing

Run it and watch it — Demo 2 and the videos

THE PROGRAM — RUN IT STEP BY STEP

- **Demo 2 — the exit check of HabitHub's Test Plan v1** — Lists the five types of testing of HabitHub's Test Plan v1 with who runs each and when, then evaluates the four exit criteria of this version with the team's numbers on Friday and on Sunday, and decides whether the version may be tagged.
- Open the module, Lesson 2: ▶️ Start, ❚❚ Pause, step through, read the trace; download `d2_exit_gate.py`

THE BEST SHORT VIDEOS FOR THIS LESSON



Software Testing Explained in 100 Seconds

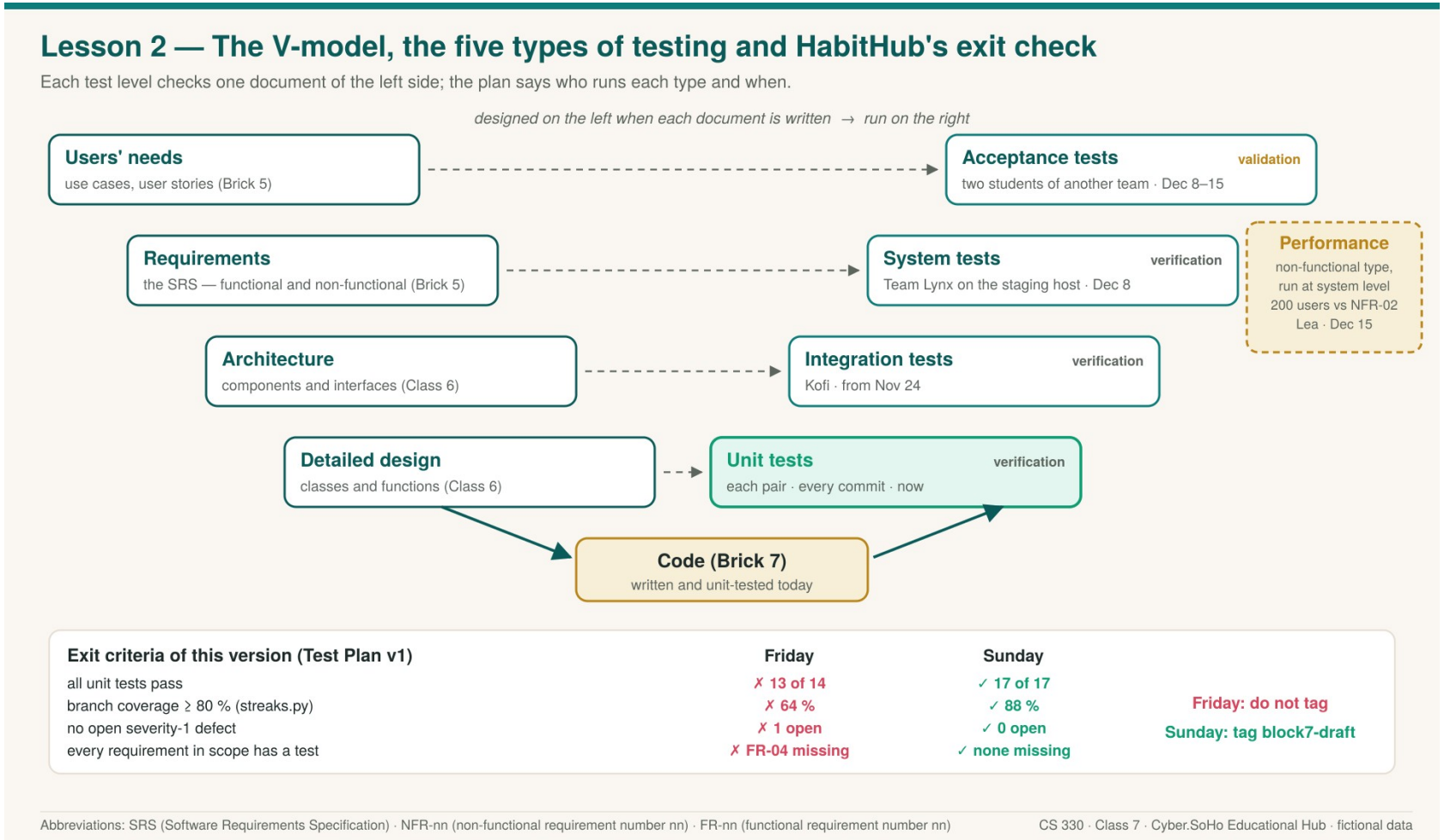
Fireship · 2:16



Verification & Validation - Georgia Tech - Software Development Process

Udacity (Georgia Tech course) · length unverified

Figure 2 — The V-model, the five types of testing and the exit check



Each test level checks one document of the left side; performance testing is a non-functional type run at system level. Below: HabitHub's four exit criteria on Friday (no tag) and Sunday (tag).

Testing from the specification only

- **Black-box testing**: test cases from the specification only — inputs and expected outputs, code not read
- Exhaustive testing is impossible: two 32-bit inputs = 2^64 ≈ 1.8 × 10^19 combinations ≈ 585 years at 10^9 tests per second
- **Test case**: identifier · input and preconditions · **expected result** written before the run · actual · status

3

Black-box design

0:30–0:31

Part C of the hour

Equivalence partitioning

- **Equivalence class** (partition): input values the specification treats the same way
- **Equivalence partitioning**: valid and invalid classes per input; one representative value per class
- StudyBuddy **FR-04** (functional requirement 04): group 2–6 → ≤ 1 (invalid) · 2–6 (valid) · ≥ 7 (invalid) · not a whole number (invalid)
- Hours ahead 2–336 → < 2 too soon (invalid) · 2–336 (valid) · > 336 too far (invalid)
- At least one invalid class per input · test invalid classes one at a time

3

Black-box design

0:31–0:33

Part C of the hour

Boundary value analysis

- **Boundary value analysis**: test on each edge and next to it — comparison defects (> vs >=) appear there
- Two-value: boundary + nearest outside → group 2–6: 1, 2, 6, 7
- Three-value: + nearest inside → 1, 2, 3, 5, 6, 7 · hours 2–336: 1, 2, 3, 335, 336, 337
- Numbers corner: 1–50 copies → 0, 1, 2, 49, 50, 51 · “at least 10” → 9, 10, 11

3

Black-box design

0:33–0:34

Part C of the hour

Decision tables

- **Decision table**: conditions and actions in rows; one column (**rule**) per combination
- 3 yes-or-no conditions → at most 2 × 2 × 2 = 8 rules; same action + “does not matter” → merge with “–”
- StudyBuddy: size valid? time valid? overlap? → 4 rules: refuse (size) · refuse (time) · refuse (overlap) · book
- Each rule → at least one test case

3

Black-box design

0:34–0:35

Part C of the hour

Error guessing

- **Error guessing**: experience-based cases for typical mistakes
- Empty · zero · negative · very large · text in a number field · leading or trailing spaces
- Double submission · February 29 · clock change · time zones
- Keep a team checklist; add a line after every surprising defect

3

Black-box design

0:35–0:36

Part C of the hour

Real-life example — StudyBuddy (Team Lynx)

THE TASK

Derive the equivalence classes and the boundary values of the booking rule, write them as black-box test cases in the Lab 07 sheet format, add the decision table of the booking checks and two error-guessing cases, then run every case against the team's first implementation.

HOW IT RELATES TO THE LESSON

Applies Lesson 3: equivalence partitioning, three-value boundary value analysis, a decision table and error guessing — all from the specification, without reading the code.

WHERE ITS DELIVERABLES GO — LABS ALREADY SCHEDULED

- **Lab 07, Step 3 — template section 4 or the 07-blackbox sheet —** Black-box tables for two use cases, with invalid classes and boundaries
- **Lab 07, Step 6 (run the black-box cases at home, open an issue per failure) —** Actual results and status filled in; the failure recorded as a defect
- **Lab 07 allows “a decision table instead of partitions” — section 0, tools and techniques —** Optional technique: the decision table
- **Lab 08 (summary only): system-level cases run by hand —** The double-submission case, run in the running product

Run it and watch it — Demo 3 and the videos

THE PROGRAM — RUN IT STEP BY STEP

- **Demo 3 — black-box tests for StudyBuddy bookings** — Derives the three-value boundary values of **FR-04** (functional requirement 04) for the group size and the hours ahead, adds the typical case, the overlap rule of the decision table and an error-guessing case, writes the expected result of every case from the specification, then runs the 15 cases against Jonas's `can_book()`.
- Open the module, Lesson 3: ▶️ Start, ❚❚ Pause, step through, read the trace; download `d3_black_box.py`

THE BEST SHORT VIDEOS FOR THIS LESSON



Boundary Value Analysis and Equivalence Partitioning: Software Testing Tutorial

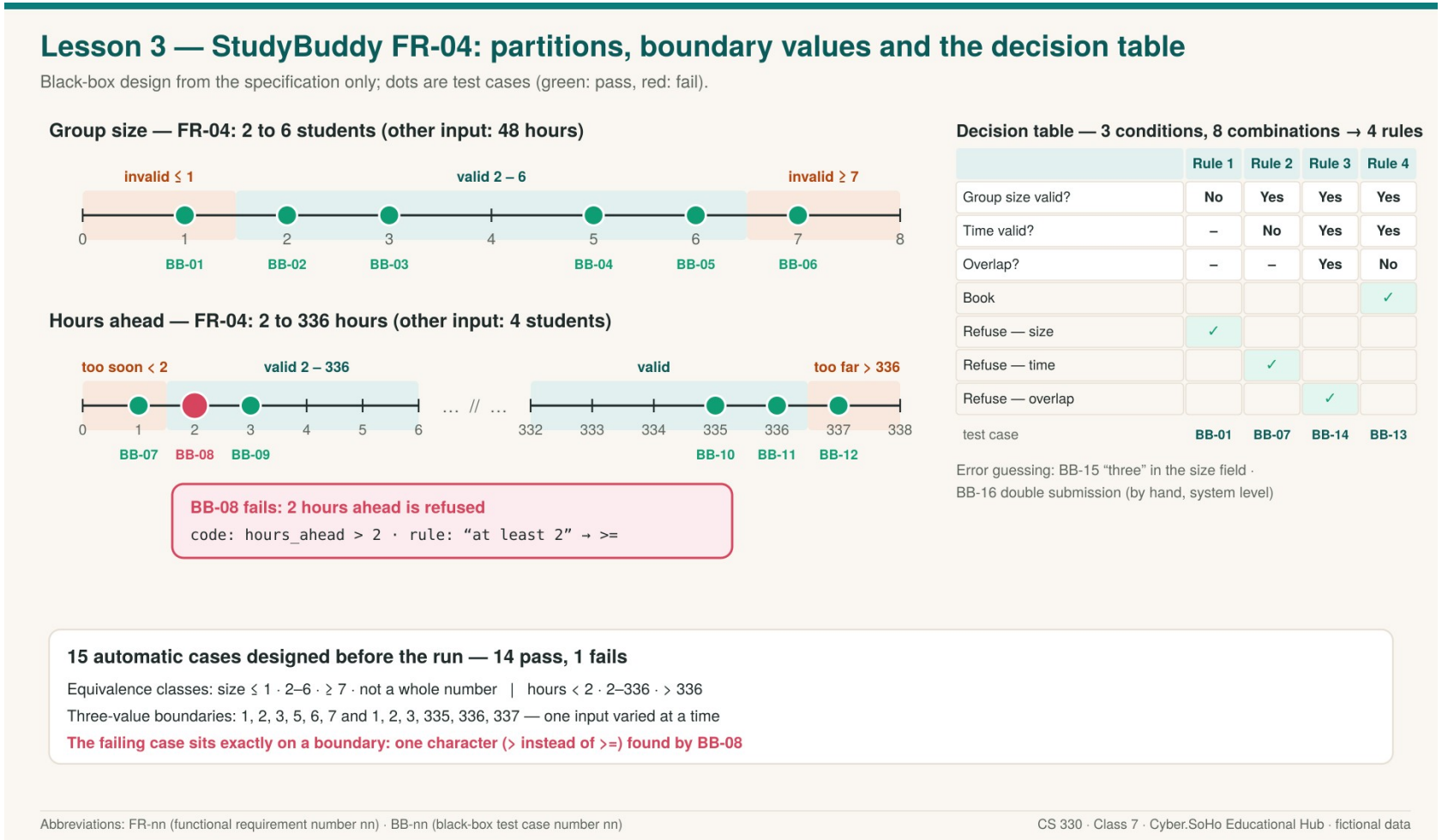
Guru99 · length unverified



What is Decision Table Testing? Software Testing Tutorial

Guru99 · length unverified

Figure 3 — Partitions, boundary values and the decision table



StudyBuddy's two inputs split into valid and invalid classes; the dots are the three-value boundary cases. The only failure, **BB-08** (black-box test case 08), sits on the lower edge of the hours.

Testing from the code: coverage

- **White-box testing** (structure-based): test cases from the code's structure — statements, branches, paths
- **Statement coverage** = executed statements ÷ statements (34 of 40 → 85 %)
- **Branch coverage** = executed decision outcomes ÷ outcomes (each `if`: true and false)
- 100 % branch ⇒ 100 % statement, not the reverse: `if x > 0: y = 1` with x = 5 → 100 % statement, 50 % branch
- Coverage shows what was not tested — not that the results were checked

4

White-box design and cyclomatic complexity

0:36–0:37

Part C of the hour

The flow graph

- **Flow graph**: **nodes** = statement blocks or single decisions · **edges** = possible transfers of control
- A decision node has two edges: yes and no
- Compound condition `a and b` → one node per simple condition (the second part runs only if the first is true)
- SafeCampus `triage_level()`: 9 nodes, 12 edges

4

White-box design and cyclomatic complexity

0:37–0:38

Part C of the hour



Cyclomatic complexity — three ways to compute it

- **Cyclomatic complexity V(G)** (cyclomatic complexity) (McCabe, 1976) — three computations that must agree:
- E − N + 2 (edges − nodes + 2): 12 − 9 + 2 = 5
- simple decisions + 1: 4 + 1 = 5 · regions of the graph (outside included): 5
- **V(G)** (cyclomatic complexity) = number of **linearly independent paths** = size of the **basis path set** = number of basis-path tests
- Upper limit for the tests needed for full branch coverage · McCabe's suggested limit per module: 10

4

White-box design and cyclomatic complexity

0:38–0:40

Part C of the hour

Basis path testing

- **Basis path testing**: draw the flow graph · compute **V(G)** (cyclomatic complexity) · choose that many independent paths · inputs + expected result per path · run and measure coverage
- Baseline method: the most common path first, then flip one decision at a time
- triage_level: baseline “normal” + 4 flips → 5 paths, 5 tests (6 complete paths; the 6th is a combination)
- Team Heron: 2 tests → 71.4 % statements, 66.7 % branches · 5 basis-path tests → 100 % and 100 %
- Tools: coverage.py with the branch option · JaCoCo · Jest

4

White-box design and cyclomatic complexity

0:40–0:41

Part C of the hour

🎯 Active Break — Fun-quiz: Three quick questions — vote with your hands

ACTIVE BREAK · FUN-QUIZ · 0:41–0:45

Answer each question by raising one to four fingers; the professor gives the one-sentence truth after each vote.

1. A flow graph has 13 edges and 10 nodes. What is **V(G)** (cyclomatic complexity)?
(1) 3 (2) 5 (3) 23 (4) 25

2. Valid input range 1–100. Which are the three-value boundary values?
(1) 1, 50, 100 (2) 0, 1, 2, 99, 100, 101 (3) 1, 100 (4) −1, 0, 100, 101

3. `if x > 0: y = 1` is tested only with x = 5. Branch coverage?
(1) 100 % (2) 75 % (3) 50 % (4) 0 %

Numbers corner — the formulas of today

Cyclomatic complexity

V(G) (cyclomatic complexity) = E − N + 2 = decisions + 1 = regions

11 edges, 9 nodes → 11 − 9 + 2 = 4

Statement coverage

executed statements ÷ statements × 100

34 of 40 → 85 %

Branch coverage

executed decision outcomes ÷ outcomes × 100

4 of 6 → 66.7 %

Defect density

defects ÷ **KLOC** (thousands of lines of code)

12 defects in 3,000 lines → 4 per KLOC (thousands of lines of code)

Boundary values

outside, on and inside each edge

1–50 copies → 0, 1, 2, 49, 50, 51

Mid-Term, November 17 — Part B asks for exactly these computations. Practise them at home in /docs/numbers.md (optional, never graded).

Real-life example — SafeCampus (Team Heron)

THE TASK

Draw the flow graph of `triage_level()`, compute **V(G)** (cyclomatic complexity) in three ways, choose the basis paths with the baseline method, write one test per path with its expected result, and measure statement and branch coverage before and after.

HOW IT RELATES TO THE LESSON

Applies Lesson 4: flow graph with a split compound condition, cyclomatic complexity, basis path testing, statement and branch coverage.

WHERE ITS DELIVERABLES GO — LABS ALREADY SCHEDULED

- **Lab 07, Step 4 — template section 5 —** The flow graph (Mermaid text or a photo), **V(G)** (cyclomatic complexity) computed three ways, basis paths mapped to tests
- **Lab 07, Step 2 — template section 3 —** Statement and branch coverage, with the lines that were not covered and why
- **Lab 07, Step 8 — template section 8 — V(G)** (cyclomatic complexity) in the numbers table

Run it and watch it — Demo 4 and the videos

THE PROGRAM — RUN IT STEP BY STEP

- **Demo 4 — basis paths and coverage of `triage_level()`** — Describes the flow graph of `triage_level()` as data, computes **V(G)** (cyclomatic complexity) with two formulas, checks that the five basis paths of the baseline method are independent and use all twelve edges, runs the tests, and measures statement and branch coverage for the team's two tests and for the five basis-path tests.
- Open the module, Lesson 4: ▶️ Start, ❚❚ Pause, step through, read the trace; download `d4_basis_paths.py`

THE BEST SHORT VIDEOS FOR THIS LESSON



Control Flow Graphs - Georgia Tech - Software Development Process

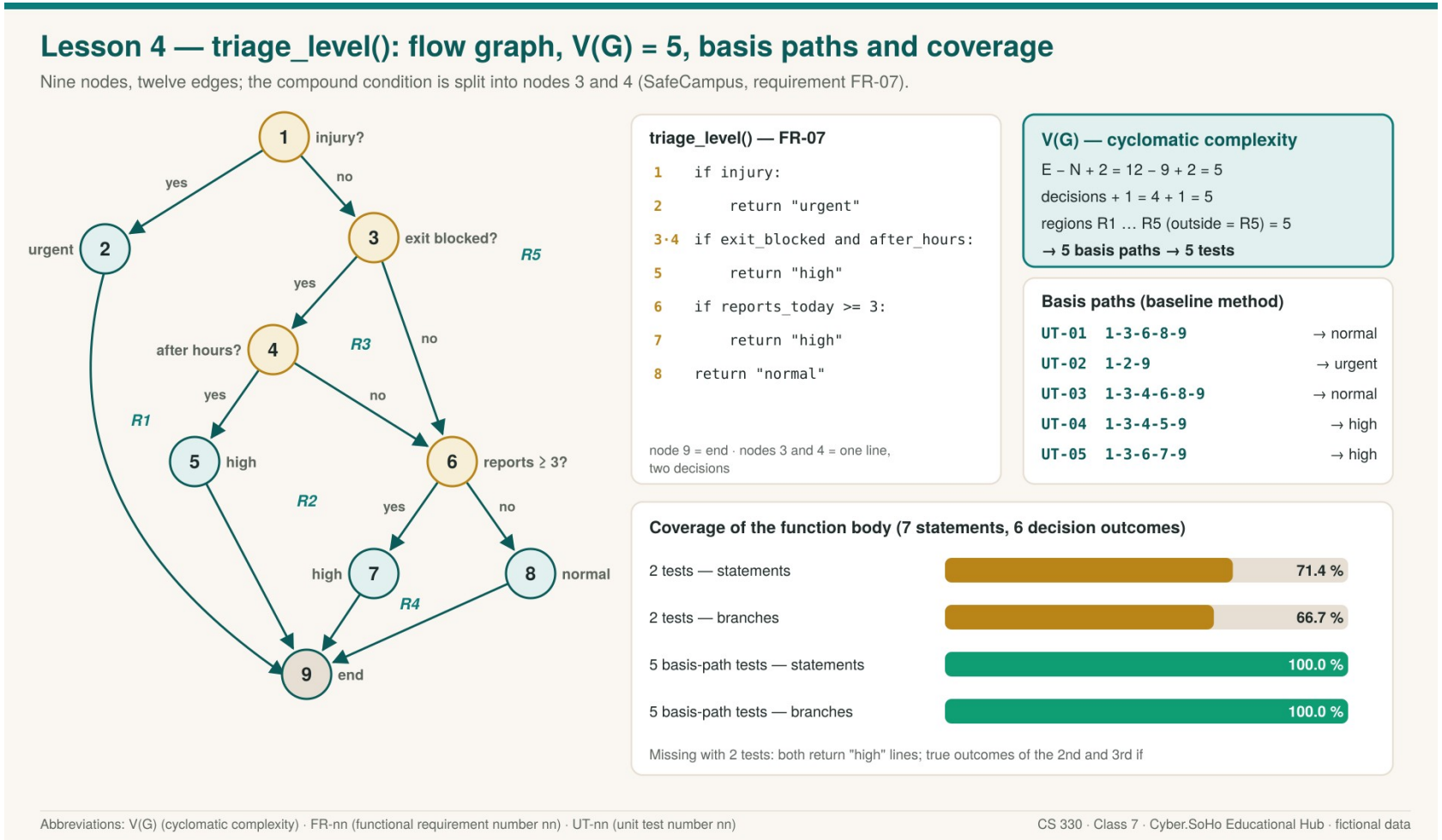
Udacity (Georgia Tech course) · length unverified



Statement Coverage - Georgia Tech - Software Development Process

Udacity (Georgia Tech course) · length unverified

Figure 4 — Flow graph, V(G) (cyclomatic complexity) and coverage of triage_level()



Nine nodes and twelve edges give **V(G)** (cyclomatic complexity) = 5 in three ways; five basis paths, five tests; coverage rises from 71.4 % and 66.7 % to 100 %.

Test-Driven Development: red, green, refactor

- **TDD (Test-Driven Development)**: the test is written before the code, in very short cycles (Kent Beck, 2002; an **XP** (Extreme Programming) practice)
- **Red**: one small test for the next behaviour — it fails, proving the behaviour is missing and the test can fail
- **Green**: the simplest code that passes all tests · **Refactor**: improve the code, all tests stay green
- QueueLess **FR-09** (functional requirement 09): red (5 min → remind) → `<= 5` → red (−1 → no) → `0 <= minutes_left <= 5` → refactor: `REMINDER_MINUTES`
- Gains: executable requirements, testable design, free regression testing · cost: tests to maintain

5

Test-first, test doubles, defects

0:45–0:48

Part D of the hour



Unit-test frameworks and the shape of a test

- **Unit-test framework** (xUnit family): finds tests, runs them, checks **assertions**, prepares **fixtures**, reports
- JUnit (Java) · pytest (Python) · Jest (JavaScript) — a starter for each in Lab 07
- **Arrange–Act–Assert**: prepare the data · call the unit once · check the expected result
- One behaviour per test, named after it: `test_no_reminder_after_slot_started`, not `test3`
- Independent tests · no dependence on today's date or the real clock

5

Test-first, test doubles, defects

0:48–0:50

Part D of the hour



Stubs, mocks and fakes

- **Test double**: an object that takes the place of a real component during a test (Meszaros, 2007)
- **Stub**: returns prepared answers — a clock fixed at 11:55 → controls the unit's inputs
- **Mock**: records the calls the unit makes → the test checks them (notify called once, order 42)
- **Fake**: a simplified working version — an in-memory list instead of the database
- Stubs in top-down integration, drivers in bottom-up · doubles do not replace integration tests (Lab 08)

5

Test-first, test doubles, defects

0:50–0:52

Part D of the hour

Debugging: from failure to corrected defect

- **Debugging**: locating and correcting the defect behind a failure — testing reveals, debugging corrects
- 1 Reproduce with the smallest input · 2 Write a failing test
- 3 Locate: hypothesis → check (read, log, **breakpoint** in the debugger; `git bisect` over commits)
- 4 Correct the cause, not the symptom · 5 Run the new test, then the whole suite · 6 Record the defect

5

Test-first, test doubles, defects

0:52–0:54

Part D of the hour

Defect reports, severity, priority and defect density

- **Defect report**: identifier · title · steps to reproduce · expected · actual · version · severity · priority · status · evidence
- **Severity**: how serious the effect is (1 critical, 2 major, 3 minor) · **Priority**: how soon to correct it (high, medium, low)
- Home-page typo: low severity, high priority · crash in an unused export: high severity, lower priority
- Life cycle: new → assigned → corrected → verified → closed (or reopened)
- **Defect density** = defects ÷ **KLOC** (thousands of lines of code) → 12 defects in 3,000 lines = 4 per thousand lines

5

Test-first, test doubles, defects

0:54–0:56

Part D of the hour

🧘 Active Break — Zen-break: Two minutes of silence, then a flash-card sprint

Lesson 5

ACTIVE BREAK · ZEN-BREAK · 0:56–1:00

Two minutes: stand up, stretch, breathe, look away from the screens — nothing to do, on purpose. Then two minutes in pairs: one partner reads a term of the week, the other gives the definition; swap after one minute.

The fourteen terms of the week: verification · validation · unit test · integration test · system test · user acceptance test · performance test · equivalence partition · boundary value · cyclomatic complexity · statement coverage · **TDD** (Test-Driven Development) · stub / mock · defect density

Real-life example — QueueLess (Team Falcon)

THE TASK

Implement the reminder rule test-first in red–green–refactor cycles; then test the reminder sender in isolation with a stub clock and a mock notifier. The mock exposes a defect — a duplicate reminder — which becomes a defect report with severity and priority, and the team updates its defect density.

HOW IT RELATES TO THE LESSON

Applies Lesson 5: the **TDD** (Test-Driven Development) cycle, a unit-test framework's shape (Arrange–Act–Assert), test doubles, debugging, the defect report, severity versus priority, defect density.

WHERE ITS DELIVERABLES GO — LABS ALREADY SCHEDULED

- **Lab 07, Step 1 —** One red → green → refactor cycle committed, with commit messages that say so
- **Lab 07, Step 2 — template section 2 —** Tests named after behaviours and traced to a functional requirement
- **Lab 07, Step 6 — template section 6 —** The defect as an issue with steps, expected, actual, severity and priority — and the test that exposed it
- **Lab 07, Step 8 — template section 8 —** Defect density with the division shown

Run it and watch it — Demo 5 and the videos

THE PROGRAM — RUN IT STEP BY STEP

- **Demo 5 — test-first reminders for QueueLess, with a stub and a mock** — Runs five **TDD** (Test-Driven Development) steps — red, green, red, green, refactor — on the reminder rule **FR-09** (functional requirement 09) with a tiny test runner; then tests the reminder sender with a stub clock and a mock notifier, lets the mock expose a duplicate reminder, writes the defect report and updates the defect density.
- Open the module, Lesson 5: ▶️ Start, ❚❚ Pause, step through, read the trace; download `d5_tdd.py`

THE BEST SHORT VIDEOS FOR THIS LESSON



Test First Development - Georgia Tech - Software Development Process

Udacity (Georgia Tech course) · length unverified



Test Doubles - Designing for and using Stubs, Fakes, Mocks, and Spies

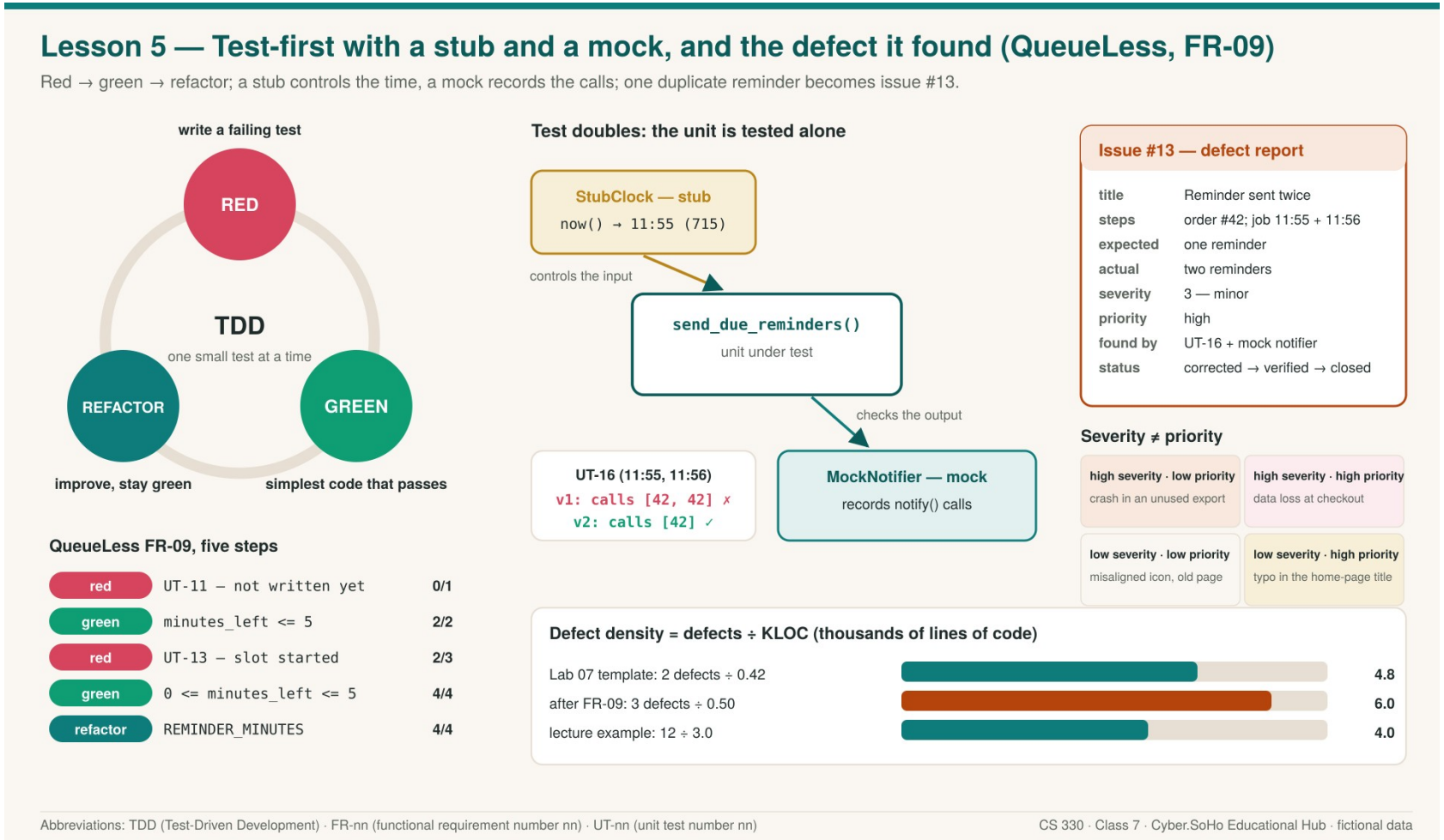
Merlin Wellington · ≈ 11 min



Software Testing Tutorial #45 - How to Write a Good Defect Report

Software Testing Mentor · length unverified

Figure 5 — TDD (Test-Driven Development), test doubles and the defect report



Five **TDD** (Test-Driven Development) steps on **FR-09** (functional requirement 09), a stub clock and a mock notifier, the defect report it produced, and the defect density before and after.

In the Studio today — Lab 07 / Brick 7

- **Step 1** · 1:10–1:25 — One **TDD** (Test-Driven Development) cycle from the starter: red → green → refactor, committed with a message that says so
- **Step 2** · 1:25–2:00 — First code of one or two core stories, in pairs; at least six unit tests named after behaviours; run the coverage tool
- **Step 3** · 2:00–2:25 — Black-box tables for two use cases: equivalence classes with at least one invalid class, and boundary values
- **Step 4** · 2:25–2:40 — White-box: the flow graph of the function with the most decisions, **V(G)** (cyclomatic complexity) three ways, basis paths mapped to tests
- **Step 5** · 2:40–2:50 — Test Plan v1: scope, the five types with who and when, tools, entry and exit criteria, defect process
- **Wrap-up** · 2:50–3:00 — The Brick 7 box, the deadline, and the exit ticket: your coverage %, your **V(G)** (cyclomatic complexity), the type of testing hardest to explain



1:10–2:50 in class · 1 h at home

Brick 7 — what to hand in, and Lab 08 next

- `/docs/07-testing.md` (sections 0–8) + code and tests in the repository, tag `block7`
- Due Sunday, December 20, 2026, 23:59 · `block7-draft` by Sunday, November 8 for feedback · 6.5 %
- Lab 08 — summary only, created separately with the Class 8 materials; it will be the last lab
- Lab 08 will run, at a small scale, the four types that Brick 7 only plans — integration, system, user acceptance, performance
- Lab 08 will close the testing of your product with a short test report



At home this week — and next Tuesday

- Finish Lab 07 at home (Steps 6–10, one hour at most).
- Read Pressman and Maxim, chapter 19 (component-level testing) and chapter 20 (integration-level testing); skim chapter 16 (reviews).
- Numbers practice, optional and never graded: **V(G)** (cyclomatic complexity), boundary values, coverage and defect density — four exercises in /docs/numbers.md.
- Read Bricks 3 and 5 again with Brick 7 beside them: do they still describe the same product?
- Class 8 — Tuesday, November 10, 2026: Mid-Term review — four rounds of retrieval practice, a mock Mid-Term and the Respondus LockDown Browser dry-run.
- Mid-Term — Tuesday, November 17, 2026 (30 %), closed book, 2 hours: Classes 1–7.



Links of the lecture (1/2)

- Testing standard 29119-1 — concepts and vocabulary
- Testing standard 29119-3 — test documentation
- Testing standard 29119-4 — test techniques
- Testing syllabus 4.0.1 — error, defect, failure; levels
- McCabe, 1976 — cyclomatic complexity
- Structured Testing, 1996 — basis paths; limit 10
- Mars Climate Orbiter release, 1999 — the unit mismatch
- Mishap board report, 1999 — the interface defect
- Pressman & Maxim, 9th edition — chapters 16, 19, 20
- Beck, Test-Driven Development — the **TDD** (Test-Driven Development) book
- Code of Ethics for Software Engineers — honest testing



Links of the lecture (2/2)

- Style Guide for Python Code — names, layout, 79 characters
- Docstring Conventions — docstrings
- Google — How to do a code review — review practice
- GitHub — Pull request reviews — reviews on GitHub
- Pro Git — commit guidelines — 50-character summary
- pytest — Python tests
- coverage.py — statement and branch coverage
- JUnit — Java tests
- Jest — JavaScript tests
- Fowler — Mocks Aren't Stubs — test doubles
- Locust — load tests
- VirusTotal — check every link first



Legal, credits and fictional data

- © 2026 Cyber.SoHo — Educational Hub · use governed by the Cyber.SoHo Legal Notice
- CS 330 is a software-engineering course: you test only your own team's code, on your own equipment
- Fictional people and data only — Legal Notice §6.7 · academic integrity — §18
- Check every link with VirusTotal link scanner before opening it — §14 and Annex C
- No vendor, standards body, platform or video channel is affiliated with Cyber.SoHo — §5

